

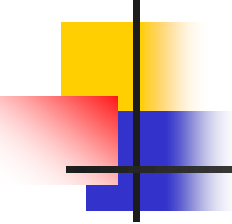
CS501

Introductory Computation for Scientists



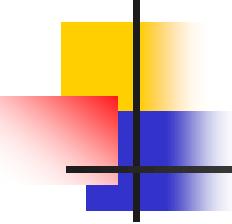
Dr. Ash Pahwa

Lesson 5: NumPy



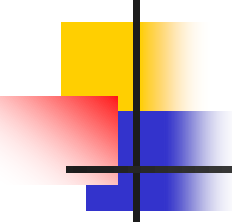
Outline

- What is NumPy?
- NumPy Vs List
- NumPy Operations
- NumPy Special Functions
- Creating Arrays and Matrices
- Array Operations
- Indexing and Slicing
- Conditional Indexing and Selection
- Copying a Matrix
- Iterate Over Arrays



What is NumPy?

- NumPy stands for Numerical Python
- NumPy is the core library for scientific computing in Python
 - Multidimensional array
 - Tools for writing array to files
 - Linear Algebra
 - Random number generation
 - Fourier Transform

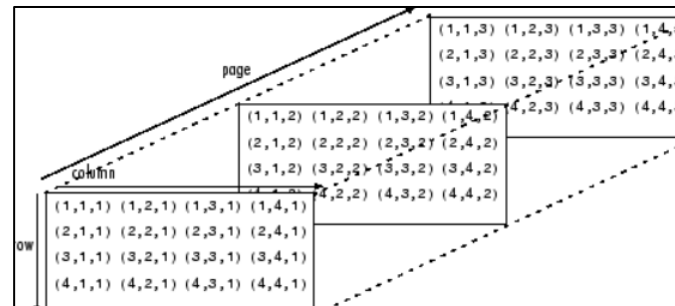


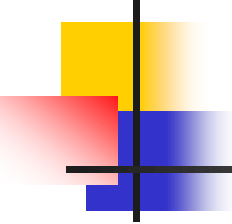
What is NumPy?

- NumPy provides
 - Used for efficient storage of multi dimensional data (big data)
 - Tools for working with arrays
 - Standard mathematical functions

Multi Dimensional Array

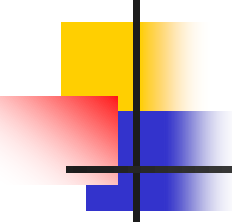
	Col 0	Col 1	Col 2
Row 0	1	2	3
Row 1	4	5	6
Row 2	7	8	9



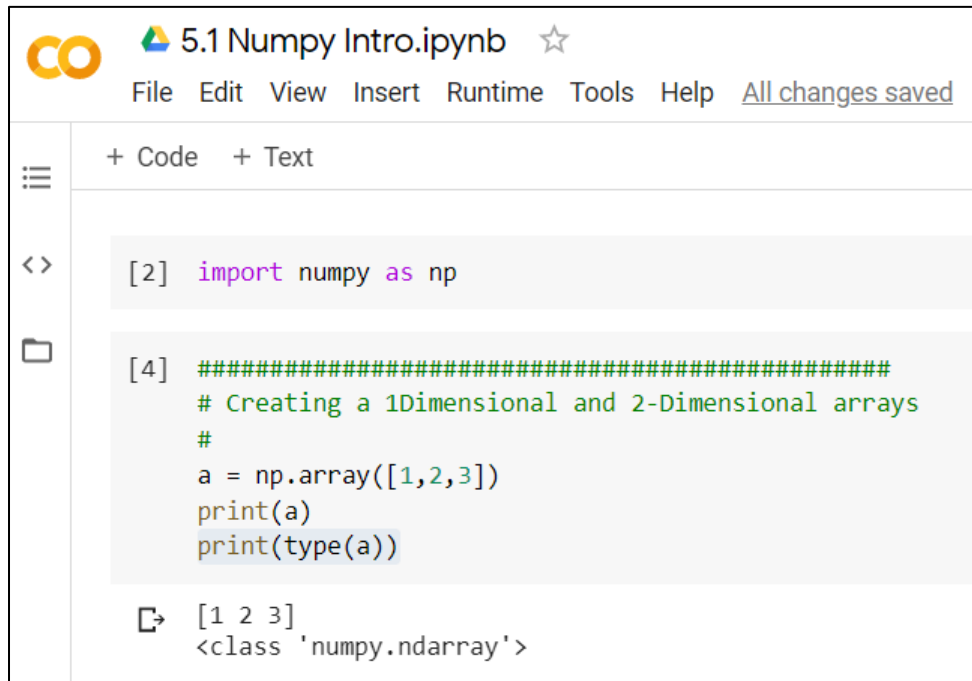


Python Data Structures

	Package • Data Structure	Create	Index Start=0	Slice	Edit
1	Core Python • String	s='abcdefghi'	s[2]	s[start:end:inc] s[2:8:2]	Immutable
2	Core Python • List	1D: l = [1,2,3] 2D: l = [1,2,list1]	1D: l[2] 2D: l[2][3]	l[start:end:inc] l[2:8:2]	Mutable
3	Core Python • Tuples	1D: t = (1,2,3) 2D: t = (1,2,list)	1D: t[2] 2D: t[2][3]	t[start:end:inc] t[2:8:2]	Immutable
4	Core Python • Dictionary	1D: d = {1:1, 2:2, 3:3 } 2D: d = {1:1, 2:2, 3:list }	Indexing Using keys		Mutable
5	Numpy • Arrays	np.array(List)	1D: a[2] 2D: a[2][3]	a[start:end:inc] a[2:8:2]	Mutable
6	Pandas • Series • DataFrame	1D: pd.Series(<index>,<data>) 2D: pd.DataFrame(<s1>,<s2>, ..)	s.loc[<index>] s.iloc[2] df.loc['index']['c1','c2']		Mutable
7	Datascience • Table	2D: t1 = Table().with_column('col name', <data>, ...)	2D: t1.select('<col name>')	2D: t1.where('<con dition>')	Mutable



Create: 1-Dimensional Array



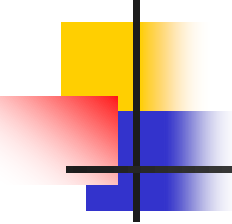
```
co 5.1 Numpy Intro.ipynb ☆
File Edit View Insert Runtime Tools Help All changes saved

+ Code + Text

[2] import numpy as np

[4] #####
    # Creating a 1Dimensional and 2-Dimensional arrays
    #
    a = np.array([1,2,3])
    print(a)
    print(type(a))

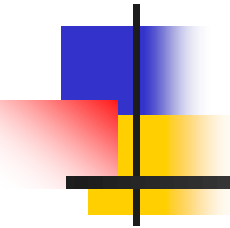
[1 2 3]
<class 'numpy.ndarray'>
```



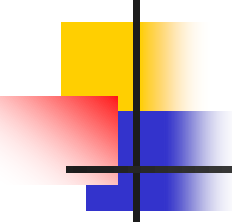
Create: 2-Dimensional Array

```
▶ a2 = np.array([(1,2,3),(4,5,6)])  
print(a2)  
print(type(a2))
```

```
↗   
[[1 2 3]  
 [4 5 6]]  
<class 'numpy.ndarray'>
```

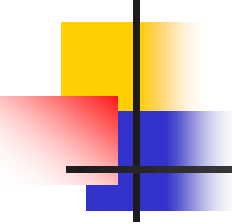


Python List and NumPy



Python Lists and NumPy

- Python has 'List' data structure
- Why do we need to have NumPy arrays?
 - Arrays take less memory space
 - Arrays are more convenient to use
 - Data retrieval from arrays is faster than lists



Python Lists

- List are created using [] square brackets
- List indexing starts with 0
- List is mutable (can be changed)

```
In [13]: items = [1,3,0,4,1]
         items
Out[13]: [1, 3, 0, 4, 1]

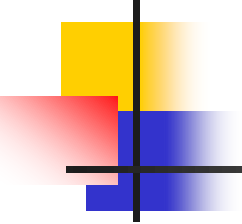
In [14]: len(items)
Out[14]: 5

In [15]: sum(items)
Out[15]: 9

In [16]: items[0]
Out[16]: 1

In [17]: items[1]=9
         items
Out[17]: [1, 9, 0, 4, 1]

In [18]: items[1:3]
Out[18]: [9, 0]
```



Arrays are More Convenient to Use

- To add elements of a list
 - We need a 'for' loop
- Arrays can be added without loops

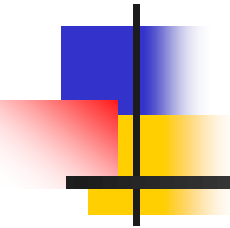
```
In [28]: #####  
         # Arrays are more convenient  
         #  
         #####  
         L1 = [1,2,3,4,5]  
         L2 = [6,7,8,9,10]  
         print(L1,L2)  
         result1 = [ (x+y) for x,y in zip(L1,L2)]  
         print(result1)  
  
         #####  
         A1 = np.array(L1)  
         A2 = np.array(L2)  
         print(A1,A2)  
         result2 = A1 + A2  
         print(result2)
```

```
[1, 2, 3, 4, 5] [6, 7, 8, 9, 10]  
[7, 9, 11, 13, 15]  
[1 2 3 4 5] [ 6  7  8  9 10]  
[ 7  9 11 13 15]
```

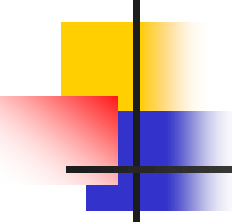
Data Retrieval from Arrays is Faster Than Lists

- Retrieval time from List
 - = 69.83 milliseconds
- Retrieval time from List
 - = 2.99 milliseconds

```
In [29]: #####  
# Data retrieval from arrays is faster than lists  
#  
import time  
SIZE=200000  
L1 = [1,2,3,4,5]*SIZE  
L2 = [6,7,8,9,10]*SIZE  
#print(L1)  
#print(L2)  
  
#####  
start1 = time.time()  
result1 = [ (x+y) for x,y in zip(L1,L2)]  
print((time.time() - start1)*1000)  
  
#print(result1)  
  
#####  
A1 = np.array(L1)  
A2 = np.array(L2)  
#print(A1)  
#print(A2)  
  
start2 = time.time()  
result2 = A1 + A2  
print((time.time() - start2)*1000)  
  
#print(result2)  
#####  
  
69.8387622833252  
2.9916763305664062
```

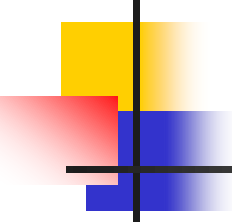


NumPy Operations



Operations

1. Find the dimension of an array
2. Find the byte size of each element
3. Find the data type of the elements
4. Find the size of an array
5. Find the shape of an array
6. Reshape: Change the number of rows and columns

- 
1. Find the dimension of an array
 2. Find the byte size of each element
 3. Find the data type of the elements
 4. Find the size of an array

```
In [3]: import numpy as np

In [7]: a = np.array([(1,2,3),(4,5,6)])
a
Out[7]: array([[1, 2, 3],
               [4, 5, 6]])

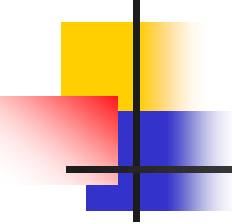
In [8]: #####
# Find Dimension of an array
print(a.ndim)
2

In [9]: #####
# Find the byte size of each element
print(a.itemsize)

#####
# Find the data type of the elements
print(a.dtype)

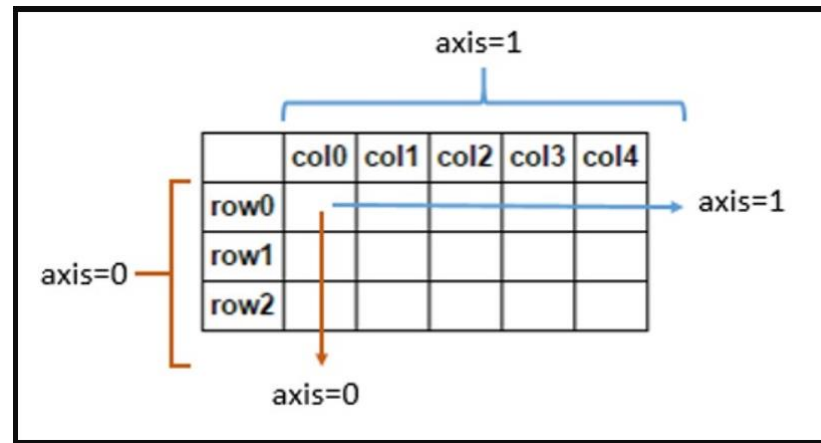
#####
# Find the size of an array
#
print(a.size)

4
int32
6
```

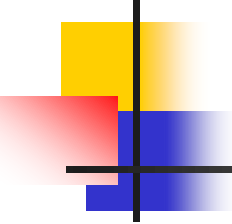
- 
5. Find the shape size of each element
 6. Reshape: Change the number of rows and columns

```
In [14]: #####  
# Find the shape of an array  
#  
a = np.array([(1,2,3),(4,5,6)])  
print(a)  
print("Dimension =",a.ndim)  
print("Shape =",a.shape)  
print("\n")  
  
#####  
# Reshape: Change number of rows and columns  
  
a = np.array([(1,2,3,4),(3,4,5,6)])  
print(a)  
print("Shape =",a.shape)  
print("\n")  
a = a.reshape(4,2)  
print(a)  
print("Shape =",a.shape)  
  
[[1 2 3]  
 [4 5 6]]  
Dimension = 2  
Shape = (2, 3)  
  
[[1 2 3 4]  
 [3 4 5 6]]  
Shape = (2, 4)  
  
[[1 2]  
 [3 4]  
 [3 4]  
 [5 6]]  
Shape = (4, 2)
```

Axis in Python

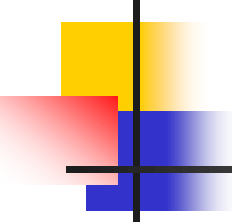


- Axis = 0 : Row 1, Row 2, Row 3, ... Row n
- Axis = 1, Column 1, Column 2, Column 3, ... Column n



Axis in Python

```
In [16]: a = np.array( [ (1,2,3),(3,4,5)])  
  
print(a)  
print("\n")  
  
print("Row wise =",a.sum(axis=0))  
  
print("\n")  
  
print("Column wise =",a.sum(axis=1))  
  
[[1 2 3]  
 [3 4 5]]  
  
Row wise = [4 6 8]  
  
Column wise = [ 6 12]
```



Square Root and Standard Deviation

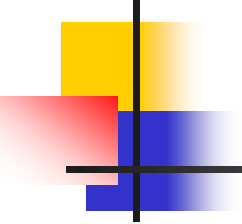
In [2]: #####

```
a = np.array( [ (1,2,3),(3,4,5)])
```

```
print(np.sqrt(a))
```

```
print(np.std(a))
```

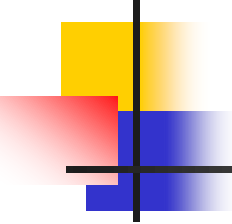
```
[[ 1.          1.41421356  1.73205081]
 [ 1.73205081  2.          2.23606798]]
1.29099444874
```



Matrix Operation

- Since 'a' and 'b' are matrices created by NumPy, we can add them with 'a+b'
- If 'a' and 'b' were lists, we cannot add them with 'a+b'. We had to write a 'for' loop to add elements of a list.
- If 'a' and 'b' were lists, 'a+b' operation will concatenate those lists

```
In [3]: #####  
  
a = np.array( [ (1,2,3),(3,4,5)])  
b = np.array( [ (1,2,3),(3,4,5)])  
print(a)  
print(b)  
  
print(a+b)  
print(a-b)  
print(a*b)  
print(a/b)  
  
[[1 2 3]  
 [3 4 5]]  
[[1 2 3]  
 [3 4 5]]  
[[ 2  4  6]  
 [ 6  8 10]]  
[[0 0 0]  
 [0 0 0]]  
[[ 1  4  9]  
 [ 9 16 25]]  
[[ 1.  1.  1.]  
 [ 1.  1.  1.]]
```



Matrix Multiplication

$$A = \begin{bmatrix} a_{11} & a_{12} & \dots & a_{1p} \\ a_{21} & a_{22} & \dots & a_{2p} \\ \dots & & & \\ a_{m1} & a_{m2} & \dots & a_{mp} \end{bmatrix} \rightarrow (m \times p)$$

$$B = \begin{bmatrix} b_{11} & b_{12} & \dots & b_{1q} \\ b_{21} & b_{22} & \dots & b_{2q} \\ \dots & & & \\ b_{p1} & b_{p2} & \dots & b_{pq} \end{bmatrix} \rightarrow (p \times q)$$

$$A * B = C$$

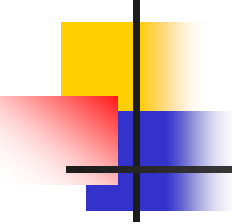
$$c_{ij} = \sum_{k=1}^p a_{ik} b_{kj}$$

This operation is valid
if number of columns of A is equal to the
number of rows of B

A Matrix dimension = $m \times p$

B Matrix dimension = $p \times q$

C Matrix dimension = $m \times q$



Matrix Multiplication

$$\begin{bmatrix} 5 & 3 \\ 8 & 2 \\ 9 & 1 \end{bmatrix} * \begin{bmatrix} 3 & 2 & 6 \\ 8 & 5 & 4 \end{bmatrix} = \begin{bmatrix} 5*3+3*8 & 5*2+3*5 & 5*6+3*4 \\ 8*3+2*8 & 8*2+2*5 & 8*6+2*4 \\ 9*3+1*8 & 9*2+1*5 & 9*6+1*4 \end{bmatrix}$$
$$= \begin{bmatrix} 15+24 & 10+15 & 30+12 \\ 24+16 & 16+10 & 48+8 \\ 27+8 & 18+5 & 54+4 \end{bmatrix} = \begin{bmatrix} 39 & 25 & 42 \\ 40 & 26 & 56 \\ 35 & 23 & 58 \end{bmatrix}$$

A Matrix dimension = 3 x 2

B Matrix dimension = 2 x 3

C Matrix dimension = 3 x 3

This operation is valid

Columns of A = Rows of B = 2

Matrix Multiplication

```
In [9]: #####  
# Matrix Multiplication  
#  
x = np.array( [[1.,2.,3.],[4.,5.,6.]] )  
y = np.array( [[6.,23.],[-1,7],[8,9]] )  
print(x)  
print(y)  
print("\n\n")  
print(x.dot(y))  
print("\n")  
print(np.dot(x,y))
```

```
[[ 1.  2.  3.]  
 [ 4.  5.  6.]]  
[[ 6. 23.]  
 [-1.  7.]  
 [ 8.  9.]]
```

```
[[ 28.  64.]  
 [ 67. 181.]]
```

```
[[ 28.  64.]  
 [ 67. 181.]]
```

Matrix Multiplication

$$\begin{bmatrix} 5 & 3 \\ 8 & 2 \\ 9 & 1 \end{bmatrix} * \begin{bmatrix} 3 & 2 & 6 \\ 8 & 5 & 4 \end{bmatrix} = \begin{bmatrix} 5*3+3*8 & 5*2+3*5 & 5*6+3*4 \\ 8*3+2*8 & 8*2+2*5 & 8*6+2*4 \\ 9*3+1*8 & 9*2+1*5 & 9*6+1*4 \end{bmatrix}$$
$$= \begin{bmatrix} 15+24 & 10+15 & 30+12 \\ 24+16 & 16+10 & 48+8 \\ 27+8 & 18+5 & 54+4 \end{bmatrix} = \begin{bmatrix} 39 & 25 & 42 \\ 40 & 26 & 56 \\ 35 & 23 & 58 \end{bmatrix}$$

```
In [10]: x = np.array( [[5., 3.],[8., 2.],[9.,1.]] )
          y = np.array( [[3.,2.,6.],[8.,5.,4.]] )
          print(x)
          print(y)

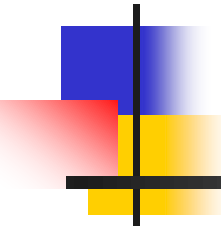
          print("\nMatrix Multiplication \n")
          print(x.dot(y))
          print("\n")
          print(np.dot(x,y))
```

```
[[ 5.  3.]
 [ 8.  2.]
 [ 9.  1.]]
[[ 3.  2.  6.]
 [ 8.  5.  4.]]
```

Matrix Multiplication

```
[[ 39.  25.  42.]
 [ 40.  26.  56.]
 [ 35.  23.  58.]]
```

```
[[ 39.  25.  42.]
 [ 40.  26.  56.]
 [ 35.  23.  58.]]
```



NumPy Special Functions

Trigonometric Functions

```
In [1]: import numpy as np
import matplotlib.pyplot as plt
```

```
In [2]: x = np.arange(0,3*np.pi,0.1)
y1 = np.sin(x)

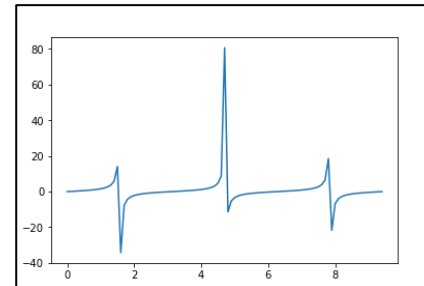
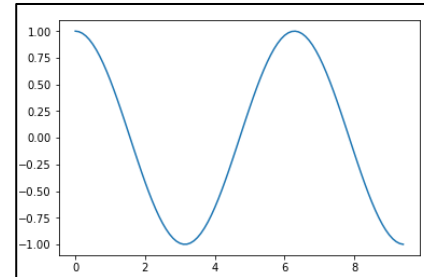
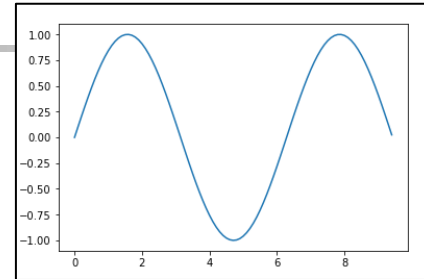
plt.plot(x,y1)
plt.show()

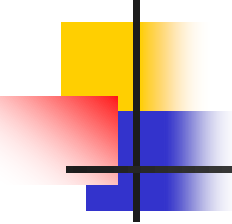
#####
y2 = np.cos(x)

plt.plot(x,y2)
plt.show()

#####
y3 = np.tan(x)

plt.plot(x,y3)
plt.show()
```





Exponential and Log Functions

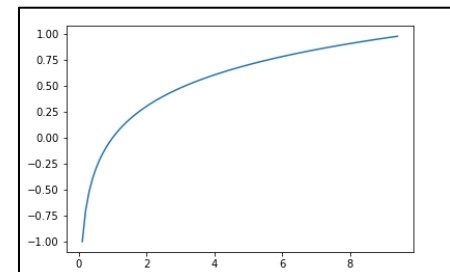
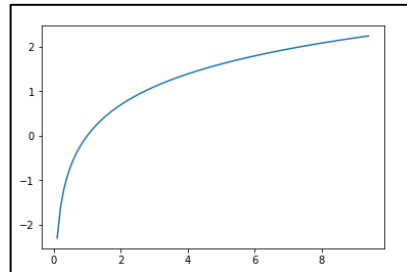
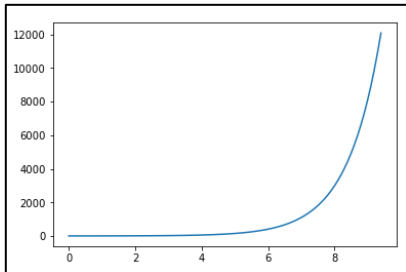
- $y = e^x$
- $y = \log_e(x)$
- $y = \log_{10}(x)$

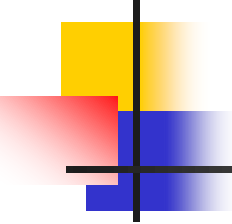
```
In [4]: #####  
ar = np.array([1,2,3])  
  
print(np.exp(ar))  
print(np.log(ar))  
print(np.log10(ar))  
  
[ 2.71828183  7.3890561 20.08553692]  
[ 0.         0.69314718 1.09861229]  
[ 0.         0.30103   0.47712125]
```

Exponential and Log Functions

- $y = e^x$
- $y = \log_e(x)$
- $y = \log_{10}(x)$

```
In [5]: #####  
  
x = np.arange(0,3*np.pi,0.1)  
y1 = np.exp(x)  
plt.plot(x,y1)  
plt.show()  
#####  
y2 = np.log(x)  
plt.plot(x,y2)  
plt.show()  
  
#####  
y3 = np.log10(x)  
plt.plot(x,y3)  
plt.show()  
#####
```





Creating Arrays and Matrices

```
In [1]: import numpy as np

In [3]: n = np.arange(0,30,2)
n
Out[3]: array([ 0,  2,  4,  6,  8, 10, 12, 14, 16, 18, 20, 22, 24, 26, 28])

In [4]: n = n.reshape(3,5)
n
Out[4]: array([[ 0,  2,  4,  6,  8],
               [10, 12, 14, 16, 18],
               [20, 22, 24, 26, 28]])

In [11]: o = np.linspace(0,4,9)
o
Out[11]: array([ 0. ,  0.5,  1. ,  1.5,  2. ,  2.5,  3. ,  3.5,  4. ])

In [12]: o.resize(3,3)
o
Out[12]: array([[ 0. ,  0.5,  1. ],
               [ 1.5,  2. ,  2.5],
               [ 3. ,  3.5,  4. ]])
```

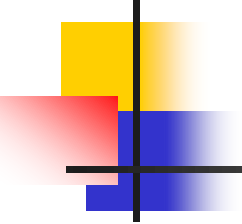
Creating Special Arrays

```
In [13]: np.ones((3,2))  
Out[13]: array([[ 1.,  1.],  
                [ 1.,  1.],  
                [ 1.,  1.]])
```

```
In [15]: np.zeros((2,3))  
Out[15]: array([[ 0.,  0.,  0.],  
                [ 0.,  0.,  0.]])
```

```
In [16]: np.eye(3)  
Out[16]: array([[ 1.,  0.,  0.],  
                [ 0.,  1.,  0.],  
                [ 0.,  0.,  1.]])
```

```
In [18]: y = [4,5,6]  
         np.diag(y)  
Out[18]: array([[4, 0, 0],  
                [0, 5, 0],  
                [0, 0, 6]])
```



Creating Special Arrays

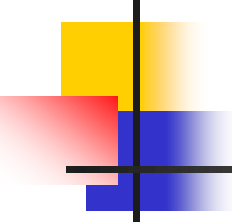
```
In [19]: np.array([1,2,3]*3)
Out[19]: array([1, 2, 3, 1, 2, 3, 1, 2, 3])

In [20]: np.repeat([1,2,3],3)
Out[20]: array([1, 1, 1, 2, 2, 2, 3, 3, 3])

In [22]: p = np.ones([2,3],int)
         p
Out[22]: array([[1, 1, 1],
               [1, 1, 1]])

In [23]: np.vstack([p,2*p])
Out[23]: array([[1, 1, 1],
               [1, 1, 1],
               [2, 2, 2],
               [2, 2, 2]])

In [24]: np.hstack([p,2*p])
Out[24]: array([[1, 1, 1, 2, 2, 2],
               [1, 1, 1, 2, 2, 2]])
```



Array Operations

```
In [39]: x=np.array([1,2,3])  
         print(x)  
         y=np.array([4,5,6])  
         print(y)
```

```
[1 2 3]  
[4 5 6]
```

```
In [40]: x+y
```

```
Out[40]: array([5, 7, 9])
```

```
In [41]: x*y
```

```
Out[41]: array([ 4, 10, 18])
```

```
In [42]: x**2
```

```
Out[42]: array([1, 4, 9])
```

```
In [44]: z=np.array([y,y**2])  
         z
```

```
Out[44]: array([[ 4,  5,  6],  
               [16, 25, 36]])
```

```
In [45]: z.shape
```

```
Out[45]: (2, 3)
```

```
In [46]: z.T
```

```
Out[46]: array([[ 4, 16],  
               [ 5, 25],  
               [ 6, 36]])
```

```
In [47]: z.T.shape
```

```
Out[47]: (3, 2)
```

Array Operations

```
In [44]: z=np.array([y,y**2])
          z
Out[44]: array([[ 4,  5,  6],
               [16, 25, 36]])

In [45]: z.shape
Out[45]: (2, 3)

In [46]: z.T
Out[46]: array([[ 4, 16],
               [ 5, 25],
               [ 6, 36]])

In [47]: z.T.shape
Out[47]: (3, 2)
```

```
In [48]: z.dtype
Out[48]: dtype('int32')

In [49]: z=z.astype('f')
          z.dtype
Out[49]: dtype('float32')

In [52]: a=np.array([-4,-2,1,3,5])
          a
Out[52]: array([-4, -2,  1,  3,  5])

In [53]: a.sum()
Out[53]: 3

In [54]: a.max()
Out[54]: 5

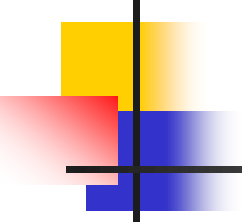
In [55]: a.min()
Out[55]: -4

In [56]: a.mean()
Out[56]: 0.59999999999999998

In [57]: a.std()
Out[57]: 3.2619012860600183

In [58]: a.argmax()
Out[58]: 4

In [59]: a.argmin()
Out[59]: 0
```



Indexing and Slicing

- $(0, 5) = 1, 2, 3, 4$
- $(0, 5] = 1, 2, 3, 4, 5$
- $[0, 5) = 0, 1, 2, 3, 4$
- $[0, 5] = 0, 1, 2, 3, 4, 5$

- In Python when we use colon operator
 - This $[n1:n2]$ (Python) **means** $[n1:n2)$ (Math)

```
In [26]: s = np.arange(13)**2
s
Out[26]: array([ 0,  1,  4,  9, 16, 25, 36, 49, 64, 81, 100, 121, 144], dtype=int32)

In [27]: s[0], s[4], s[0:3]
Out[27]: (0, 16, array([0, 1, 4], dtype=int32))

In [28]: s[1:5]
Out[28]: array([ 1,  4,  9, 16], dtype=int32)

In [29]: s[-4:]
Out[29]: array([ 81, 100, 121, 144], dtype=int32)

In [31]: s[-5::-2]
Out[31]: array([64, 36, 16,  4,  0], dtype=int32)
```

Indexing and Slicing

- $(0, 5) = 1, 2, 3, 4$
- $(0, 5] = 1, 2, 3, 4, 5$
- $[0, 5) = 0, 1, 2, 3, 4$
- $[0, 5] = 0, 1, 2, 3, 4, 5$

- In Python when we use colon operator
 - This $[n1:n2]$ (Python) **means** $[n1:n2)$ (Math)

```
In [34]: r = np.arange(36)
         r.resize(6,6)
         r

Out[34]: array([[ 0,  1,  2,  3,  4,  5],
                [ 6,  7,  8,  9, 10, 11],
                [12, 13, 14, 15, 16, 17],
                [18, 19, 20, 21, 22, 23],
                [24, 25, 26, 27, 28, 29],
                [30, 31, 32, 33, 34, 35]])

In [35]: r[2,2]

Out[35]: 14

In [36]: r[3,3:6]

Out[36]: array([21, 22, 23])

In [37]: r[:2,:-1]

Out[37]: array([[ 0,  1,  2,  3,  4],
                [ 6,  7,  8,  9, 10]])

In [38]: r[-1,:2]

Out[38]: array([30, 32, 34])
```

Conditional Indexing And Selection

```
In [39]: r = np.arange(36)
         r.resize(6,6)
         r

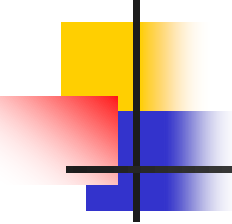
Out[39]: array([[ 0,  1,  2,  3,  4,  5],
                [ 6,  7,  8,  9, 10, 11],
                [12, 13, 14, 15, 16, 17],
                [18, 19, 20, 21, 22, 23],
                [24, 25, 26, 27, 28, 29],
                [30, 31, 32, 33, 34, 35]])

In [40]: r[r>30]

Out[40]: array([31, 32, 33, 34, 35])

In [41]: r[r>30]=30
         r

Out[41]: array([[ 0,  1,  2,  3,  4,  5],
                [ 6,  7,  8,  9, 10, 11],
                [12, 13, 14, 15, 16, 17],
                [18, 19, 20, 21, 22, 23],
                [24, 25, 26, 27, 28, 29],
                [30, 30, 30, 30, 30, 30]])
```



Iterate Over Arrays

```
In [43]: test = np.random.randint(0,10,(4,3))  
test
```

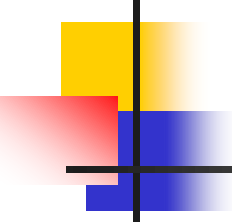
```
Out[43]: array([[7, 2, 9],  
               [6, 3, 0],  
               [5, 0, 0],  
               [9, 5, 5]])
```

```
In [44]: for row in test:  
         print(row)
```

```
[7 2 9]  
[6 3 0]  
[5 0 0]  
[9 5 5]
```

```
In [45]: for i in range(len(test)):  
         print(test[i])
```

```
[7 2 9]  
[6 3 0]  
[5 0 0]  
[9 5 5]
```



Iterate Over Arrays

```
In [46]: for i,row in enumerate(test):  
         print('row',i,'is',row)
```

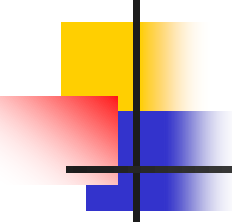
```
row 0 is [7 2 9]  
row 1 is [6 3 0]  
row 2 is [5 0 0]  
row 3 is [9 5 5]
```

```
In [49]: test2 = test**2  
         test2
```

```
Out[49]: array([[49,  4, 81],  
               [36,  9,  0],  
               [25,  0,  0],  
               [81, 25, 25]], dtype=int32)
```

```
In [50]: for i,j in zip(test,test2):  
         print(i,'+',j,'=',i+j)
```

```
[7 2 9] + [49  4 81] = [56  6 90]  
[6 3 0] + [36  9  0] = [42 12  0]  
[5 0 0] + [25  0  0] = [30  0  0]  
[9 5 5] + [81 25 25] = [90 30 30]
```



Iterate Over 2D Array

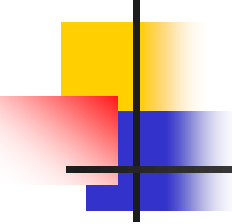
```
[ ] #Iterate over 2D Array
```

```
[12] i = np.array([[10,22,13],[24,54,67]])  
      print(i)
```

```
↳ [[10 22 13]  
    [24 54 67]]
```

```
▶ for row in i:  
    for column in row:  
        print(column, end=' ')  
    print()
```

```
↳ 10 22 13  
   24 54 67
```



Summary

- What is NumPy?
- NumPy Vs List
- NumPy Operations
- NumPy Special Functions
- Import NumPy
- Creating Arrays and Matrices
- Array Operations
- Indexing and Slicing
- Conditional Indexing and Selection
- Copying a Matrix
- Iterate Over Arrays